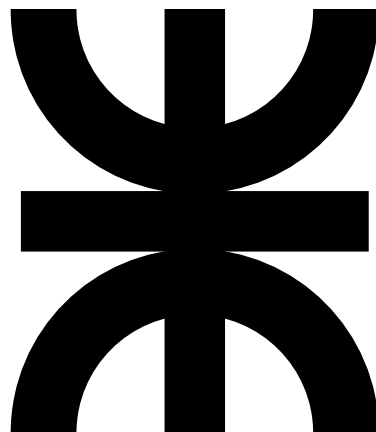


UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA
INGENIERÍA ELECTRÓNICA



Técnicas Digitales 1

Trabajo Práctico de Laboratorio N°1

Álgebra de Boole y Circuitos Combinacionales

DOCENTES : Esp.Ing. Marcelo Casasnovas
Ing Florencia Belén Molla JTP

CÁTEDRA : Técnicas Digitales 1

CURSO : 3R4

ALUMNO : Gonza Elías Noel Imanol 87669

CÓRDOBA, ARGENTINA
2026



Índice

1. Introducción	2
2. Teoría de los Códigos BCD y Exceso 3	2
2.1. Código BCD	2
2.2. Código Exceso 3	2
3. Conversor de Código BCD a Exceso 3: Diseño, Simplificación, Simulación e Implementación	2
3.1. Diseño	3
3.2. Tabla de Verdad	3
3.3. Tabla de Salidas	4
3.4. Simplificación	4
3.5. Simulación	5
3.6. Implementación	6
3.6.1. Implementación para A_1	6
3.6.2. Implementación para B_1	7
3.6.3. Implementación para C_1	7
3.6.4. Cantidad de integrados utilizados	7
3.6.5. Circuito a implementar	8
4. Comparador Binario: Diseño, Simplificación, Simulación e Implementación	8
4.0.1. Diseño	9
4.0.2. Tabla de verdad	10
4.0.3. Simplificación	10
4.0.4. Simulación	12
4.0.5. Implementación	13
4.0.6. Implementación con compuertas NAND y NOT	13
4.0.7. Circuito a implementar	14
Bibliografía	15



1. Introducción

En este laboratorio vamos a aplicar los conceptos aprendidos en las unidades 1 y 2 de la materia. En el primer punto se diseñará un conversor que transforma un número en código BCD a su equivalente en Exceso 3. De aquí desarrollaremos la tabla de verdad y su correspondiente mapa de Karnaugh para poder simplificar las funciones. Además, se llevará a la práctica utilizando el simulador Falstad y luego lo implementaremos en una protoboard junto con el MiniLab.

Para el siguiente ejercicio haremos un comparador binario que evaluará tres situaciones distintas: determinar si un número A es mayor que un número B , si es menor o si ambos son iguales. Para esto, se utilizarán tres salidas que indicarán el resultado según el orden establecido. Al igual que en el caso anterior, se realizará la simplificación de la función utilizando el mapa de Karnaugh y se llevará a la práctica utilizando un simulador y una protoboard.

2. Teoría de los Códigos BCD y Exceso 3

2.1. Código BCD

En informática, BCD es un código que se utiliza para representar números decimales en código binario. En BCD, o decimal codificado en binario, cada número decimal del 0 al 9 es representado por su equivalente en binario en 4 bits.

2.2. Código Exceso 3

El código Exceso 3 para un número decimal se obtiene de la misma forma que el BCD, excepto que se suma el número 3 a cada dígito decimal antes de modificarlo en binario.

Por ejemplo, para codificar el número decimal 4 en código Exceso 3, primero debemos sumar 3:

$$4 + 3 = 7$$

Luego, el número 7 se codifica en su equivalente binario de 4 bits:

$$7_{10} = 0111_2$$

Por lo tanto, el número decimal 4 en código Exceso 3 se representa como:

$$4_{10} \rightarrow 0111_2$$

3. Conversor de Código BCD a Exceso 3: Diseño, Simplificación, Simulación e Implementación

Este conversor cuenta con 4 entradas representadas por interruptores On-Off, donde cada interruptor codifica un dígito binario. Por ejemplo, si todos los interruptores están en posición Off, la entrada será 0000 en binario.

La salida del conversor se representa mediante 4 LEDs, los cuales indican el número en código Exceso 3. Por ejemplo, si la entrada es 0000, correspondiente al decimal 0 en BCD, la salida será 0011, equivalente al decimal 3 en Exceso 3.

$$0000_{BCD} \rightarrow 0011_{XS3}$$

3.1. Diseño

Así será el diagrama para empezar a tener una idea de cómo será el diseño. Las entradas $A0$, $B0$, $C0$ y $D0$ serán representadas mediante interruptores, mientras que las salidas $A1$, $B1$, $C1$ y $D1$ serán representadas mediante LEDs.

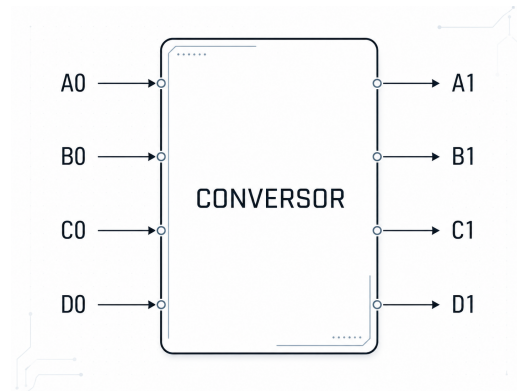


Figura 1: Diagrama general del conversor BCD a Exceso 3.

3.2. Tabla de Verdad

En la siguiente tabla se muestran las combinaciones de 4 bits tanto en BCD como en Exceso 3. También se indican las funciones de salida $A1$, $B1$, $C1$ y $D1$, las cuales tomarán los valores 1 o 0 dependiendo de las entradas BCD $A0$, $B0$, $C0$ y $D0$.

Decimal	Binario	BCD	Exceso 3
0	0000	0000	0011
1	0001	0001	0100
2	0010	0010	0101
3	0011	0011	0110
4	0100	0100	0111
5	0101	0101	1000
6	0110	0110	1001
7	0111	0111	1010
8	1000	1000	1011
9	1001	1001	1100
		$A0\ B0\ C0\ D0$	$A1\ B1\ C1\ D1$

Cuadro 1: Tabla de conversión de BCD a Exceso 3.

3.3. Tabla de Salidas

A partir de la tabla de verdad anterior, se obtienen las salidas correspondientes del conversor:

A_1	B_1	C_1	D_1
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0

Cuadro 2: Estados de salida A_1 , B_1 , C_1 y D_1 para cada entrada BCD.

3.4. Simplificación

Luego de armar los mapas de Karnaugh, llegamos a la simplificación de las siguientes funciones A_1 , B_1 , C_1 y D_1 .

		C_0D_0			
A_0B_0		00	01	11	10
00		0	0	0	0
01		0	1	1	1
11		X	X	X	X
10		1	1	X	X

$$A_1 = A_0 + B_0(C_0 + D_0)$$

		C_0D_0			
A_0B_0		00	01	11	10
00		0	1	1	1
01		1	0	0	0
11		X	X	X	X
10		0	1	X	X

$$B_1 = B_0D'_0C'_0 + B'_0(D_0 + C_0)$$

	C_0D_0			
A_0B_0	00	01	11	10
00	1	0	1	0
01	1	0	1	0
11	X	X	X	X
10	1	0	X	X

$$C_1 = C'_0D'_0 + C_0D_0 = (C_0 \oplus D_0)'$$

	C_0D_0			
A_0B_0	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	X	X	X	X
10	1	0	X	X

$$D_1 = D'_0$$

3.5. Simulación

Mediante el simulador online *Falstad*, llevamos a cabo la implementación del circuito utilizando las compuertas correspondientes para comprobar su funcionamiento. Luego, verificamos su comportamiento con la tabla de verdad asociada.

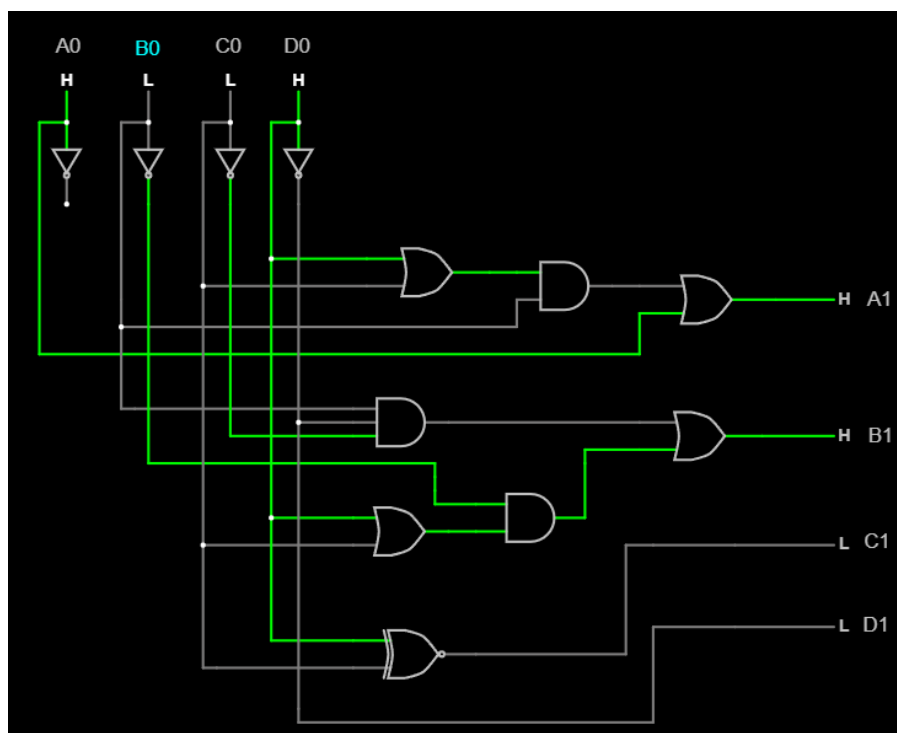


Figura 2: Simulación del conversor BCD a Exceso 3 en Falstad.

Simulación 1 en Falstad

Como podemos observar, para la combinación de entrada:

$$A0 = 1, \quad B0 = 0, \quad C0 = 0, \quad D0 = 1$$

en la salida se obtiene:

$$A1 = 1, \quad B1 = 1, \quad C1 = 0, \quad D1 = 0$$

Esto corresponde al número 9 en BCD, ya que:

$$1001_{BCD} = 9_{10}$$

Al sumarle 3 para obtener el código Exceso 3:

$$9 + 3 = 12$$

y el número 12 en binario de 4 bits es:

$$12_{10} = 1100_2$$

Por lo tanto:

$$1001_{BCD} \rightarrow 1100_{XS3}$$

Esto coincide con lo indicado por la tabla de verdad del conversor. La simulación se puede consultar en el siguiente enlace:

3.6. Implementación

Una vez pasado el circuito a la simulación, ya podemos asegurarnos de que las funciones que resultan de aplicar el mapa de Karnaugh son correctas. Ahora, por último, vamos a llevar toda la teoría a la práctica, armando todo el circuito en una placa protoboard. Para ello, veremos su funcionamiento como lo mencionamos anteriormente.

Para tener una mayor facilidad en la implementación, vamos a llevar todo el circuito a compuertas NAND y NOT.

3.6.1. Implementación para A_1

Partimos de la función simplificada:

$$A_1 = A_0 + B_0(C_0 + D_0)$$

Aplicando equivalencias de De Morgan:

$$A_1 = A_0 + B_0(\overline{\overline{C_0 + D_0}})$$

Como:

$$C_0 + D_0 = \overline{\overline{C_0} \cdot \overline{D_0}}$$

entonces:

$$A_1 = A_0 + B_0(\overline{\overline{C_0} \cdot \overline{D_0}})$$

Aplicando doble negación:

$$A_1 = \overline{\overline{A_0 + B_0 (\overline{\overline{C_0 \cdot D_0}})}} \quad \overline{\overline{C_0 \cdot D_0}}$$

Finalmente, expresado para implementación con NAND:

$$A_1 = \overline{\overline{\overline{A_0} \cdot B_0 (\overline{\overline{C_0 \cdot D_0}})}} \quad \overline{\overline{C_0 \cdot D_0}}$$

3.6.2. Implementación para B_1

Partimos de la función simplificada:

$$B_1 = B_0 \cdot \overline{D_0} \cdot \overline{C_0} + \overline{B_0} \cdot (C_0 + D_0)$$

Aplicando De Morgan en la suma $C_0 + D_0$:

$$B_1 = B_0 \cdot \overline{D_0} \cdot \overline{C_0} + \overline{B_0} \cdot (\overline{\overline{C_0 \cdot D_0}})$$

Aplicando doble negación a toda la expresión:

$$B_1 = \overline{\overline{B_0 \cdot \overline{D_0} \cdot \overline{C_0} + \overline{B_0} \cdot (\overline{\overline{C_0 \cdot D_0}})}} \quad \overline{\overline{C_0 \cdot D_0}}$$

Finalmente, expresado en forma conveniente para compuertas NAND:

$$B_1 = \overline{\overline{\overline{B_0 \cdot \overline{D_0} \cdot \overline{C_0}} \cdot \overline{B_0} \cdot (\overline{\overline{C_0 \cdot D_0}})}} \quad \overline{\overline{C_0 \cdot D_0}}$$

3.6.3. Implementación para C_1

Partimos de la función simplificada:

$$C_1 = \overline{C_0} \cdot \overline{D_0} + C_0 \cdot D_0$$

Aplicando doble negación:

$$C_1 = \overline{\overline{\overline{\overline{C_0} \cdot \overline{D_0}} + C_0 \cdot D_0}}$$

Finalmente, aplicando De Morgan para llevarlo a compuertas NAND:

$$C_1 = \overline{\overline{\overline{\overline{C_0} \cdot \overline{D_0}} \cdot \overline{C_0 \cdot D_0}}}$$

3.6.4. Cantidad de integrados utilizados

En total usaremos:

5 compuertas NOT

7 compuertas NAND de 2 entradas

1 compuertas XNOR de 2 entradas

Los integrados que corresponden a las compuertas son:

- CD4069: integrado con compuertas NOT.
- CD4011: integrado con compuertas NAND de 2 entradas.
- CD4077: integrado con compuertas XNOR de 2 entradas.

Por lo tanto, utilizaremos un integrado CD4069 para las compuertas inversoras, dos integrados CD4011 para las compuertas NAND de 2 entradas y un integrado CD4077 para las compuertas XNOR.

3.6.5. Circuito a implementar

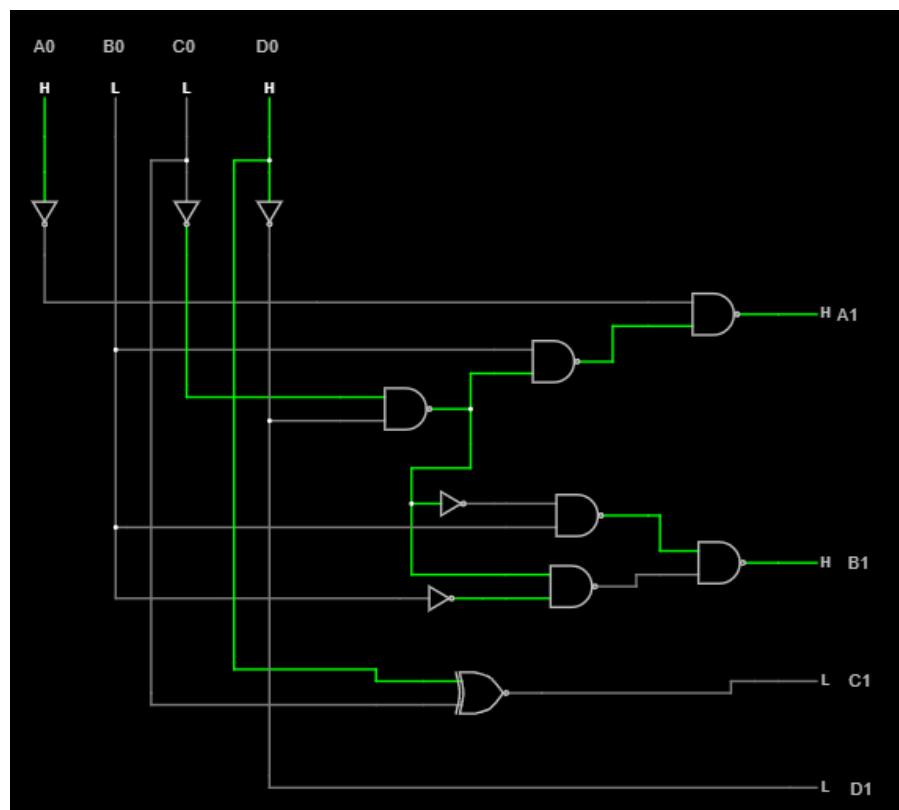


Figura 3: Circuito final implementado con compuertas NAND, NOT y XNOR.

Simplificación 1 en Falstad

4. Comparador Binario: Diseño, Simplificación, Simulación e Implementación

Para el segundo ejercicio se implementará un comparador binario de 2 bits. El circuito tendrá cuatro entradas, de las cuales dos corresponden al número A y las otras dos al número B . Estas entradas se compararán para determinar si A es mayor que B , si A es menor que B o si ambos valores son iguales.

Las entradas del comparador serán:

$$A = A_1A_0$$

$$B = B_1B_0$$

Las salidas del circuito serán S_0 , S_1 y S_2 , las cuales indicarán las siguientes condiciones:

- S_0 : indica que $A > B$.
- S_1 : indica que $A < B$.
- S_2 : indica que $A = B$.

Por lo tanto, el comportamiento esperado del comparador es el siguiente:

1. Si $A > B$:

$$S_0 = 1 \quad (\text{LED encendido})$$

$$S_1 = 0 \quad (\text{LED apagado})$$

$$S_2 = 0 \quad (\text{LED apagado})$$

2. Si $A < B$:

$$S_0 = 0 \quad (\text{LED apagado})$$

$$S_1 = 1 \quad (\text{LED encendido})$$

$$S_2 = 0 \quad (\text{LED apagado})$$

3. Si $A = B$:

$$S_0 = 0 \quad (\text{LED apagado})$$

$$S_1 = 0 \quad (\text{LED apagado})$$

$$S_2 = 1 \quad (\text{LED encendido})$$

4.0.1. Diseño

Como en el ejercicio anterior, para guiarnos mejor se realiza un diagrama en bloques con sus respectivas entradas y salidas. Esto permite visualizar de manera más clara el funcionamiento general del comparador antes de desarrollar la tabla de verdad, las funciones lógicas y la implementación física.

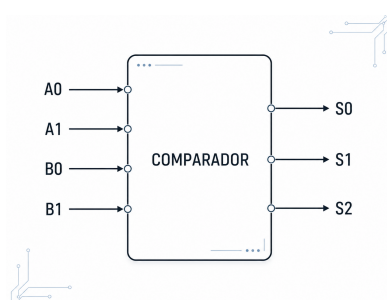


Figura 4: Diagrama en bloques del comparador binario de 2 bits.



4.0.2. Tabla de verdad

La tabla de verdad muestra las distintas combinaciones posibles de las entradas A_0 , A_1 , B_0 y B_1 , junto con los valores correspondientes de las salidas S_1 , S_2 y S_3 . Estas salidas indicarán si A es mayor que B , si A es menor que B o si ambos valores son iguales.

A_1	A_0	B_1	B_0	S_0	S_1	S_2
0	0	0	0	0	0	1
0	0	0	1	0	1	0
0	0	1	0	0	1	0
0	0	1	1	0	1	0
0	1	0	0	1	0	0
0	1	0	1	0	0	1
0	1	1	0	0	1	0
0	1	1	1	0	1	0
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	0	1
1	0	1	1	0	1	0
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	0	1

4.0.3. Simplificación

Luego de armar los mapas de Karnaugh, se obtienen las simplificaciones de las funciones lógicas correspondientes a cada salida del comparador binario.

	B_0B_1			
A_0A_1	00	01	11	10
00	0	0	0	0
01	1	0	0	0
11	1	1	0	1
10	1	1	0	0

$$S_0 = A_0\overline{B_0} + A_1\overline{B_1}(\overline{B_0} + A_0)$$

	B_0B_1			
A_0A_1	00	01	11	10
00	0	1	1	1
01	0	0	0	1
11	0	0	0	0
10	0	0	1	0

$$S_1 = \overline{A_0}B_0 + \overline{A_1}B_1(\overline{A_0} + B_0)$$

	B_0B_1			
A_0A_1	00	01	11	10
00	1	0	0	0
01	0	1	0	0
11	0	0	1	0
10	0	0	0	1

Función impar, por lo tanto no se puede simplificar:

$$\begin{aligned} S_2 = & \overline{A_0} \overline{A_1} \overline{B_0} \overline{B_1} \\ & + \overline{A_0} A_1 \overline{B_0} B_1 \\ & + A_0 A_1 B_0 B_1 \\ & + A_0 \overline{A_1} B_0 \overline{B_1} \end{aligned}$$

4.0.4. Simulación

Luego de obtener las funciones lógicas del comparador binario, se realizó la simulación del circuito para verificar su correcto funcionamiento. En la siguiente figura se puede observar el circuito implementado en el simulador.

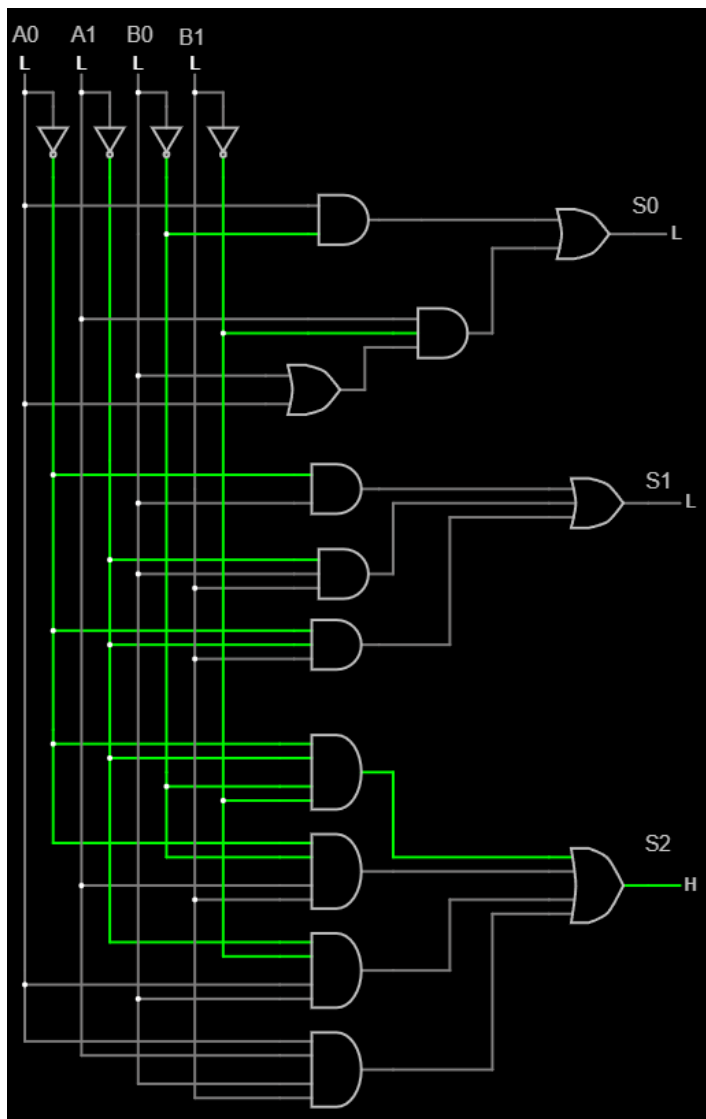


Figura 5: Simulación del comparador binario de 2 bits.

Simulación 2 en Falstad

Como se puede observar en la simulación, las entradas se encuentran en el siguiente estado:

$$A_0 = 0, \quad A_1 = 0, \quad B_0 = 0, \quad B_1 = 0$$

Por lo tanto, los valores de entrada corresponden a:

$$A = 00$$

$$B = 00$$

En este caso se cumple que:

$$A = B$$

Por este motivo, las salidas del comparador toman los siguientes valores:

$$S_0 = 0, \quad S_1 = 0, \quad S_2 = 1$$

Esto coincide con lo indicado en la tabla de verdad, ya que cuando ambos números son iguales, solamente debe activarse la salida correspondiente a la igualdad. La simulación del circuito puede observarse en el enlace correspondiente a la Simulación 2.

4.0.5. Implementación

Una vez verificado el funcionamiento mediante la simulación, se pasa a la implementación física del circuito utilizando compuertas lógicas y LEDs indicadores. Al igual que en el ejercicio anterior, cada salida del comparador se conecta a un LED para comprobar visualmente el estado lógico de cada condición.

Para facilitar la implementación física del circuito, se utilizarán compuertas lógicas equivalentes a las funciones simplificadas obtenidas mediante los mapas de Karnaugh. De esta manera, el circuito permitirá indicar si A es mayor que B , si A es menor que B o si ambos valores son iguales.

4.0.6. Implementación con compuertas NAND y NOT

Para la implementación física del comparador, se procede a transformar las funciones lógicas obtenidas a compuertas NAND y NOT.

Para S_0 :

$$S_0 = A_0 \overline{B_0} + A_1 \overline{B_1} (\overline{B_0} + A_0)$$

$$S_0 = A_0 \overline{B_0} + A_1 \overline{B_1} \cdot \overline{\overline{\overline{B_0} + A_0}}$$

$$S_0 = A_0 \overline{B_0} + A_1 \overline{B_1} \cdot \overline{(B_0 \cdot \overline{A_0})}$$

$$S_0 = \overline{\overline{\overline{A_0 \overline{B_0} + A_1 \overline{B_1} \cdot (B_0 \cdot \overline{A_0})}}}$$

$$S_0 = \overline{\overline{\overline{A_0 \overline{B_0}} \cdot \overline{\overline{\overline{A_1 \overline{B_1}} \cdot \overline{\overline{\overline{B_0 \cdot \overline{A_0}}}}}}}}$$

Para S_1 :

$$S_1 = \overline{A_0} B_0 + \overline{A_1} B_1 (\overline{A_0} + B_0)$$

$$S_1 = \overline{A_0} B_0 + \overline{A_1} B_1 \cdot \overline{\overline{\overline{\overline{A_0} + B_0}}}$$

$$S_1 = \overline{A_0} B_0 + \overline{A_1} B_1 \cdot \overline{(A_0 \cdot \overline{B_0})}$$

$$S_1 = \overline{\overline{\overline{A_0}B_0} + \overline{\overline{A_1}B_1} \cdot \overline{(A_0 \cdot \overline{B_0})}}$$

$$S_1 = \overline{\overline{\overline{A_0}B_0} \cdot \overline{\overline{A_1}B_1} \cdot \overline{(A_0 \cdot \overline{B_0})}}$$

Para S_2 : Para la salida S_2 , se observa que corresponde al caso en el que ambos números son iguales. Por lo tanto, se puede aprovechar la comparación bit a bit mediante compuertas XNOR:

$$S_2 = S_0 \odot S_1$$

donde el símbolo $\odot$ representa la operación XNOR.

4.0.7. Circuito a implementar

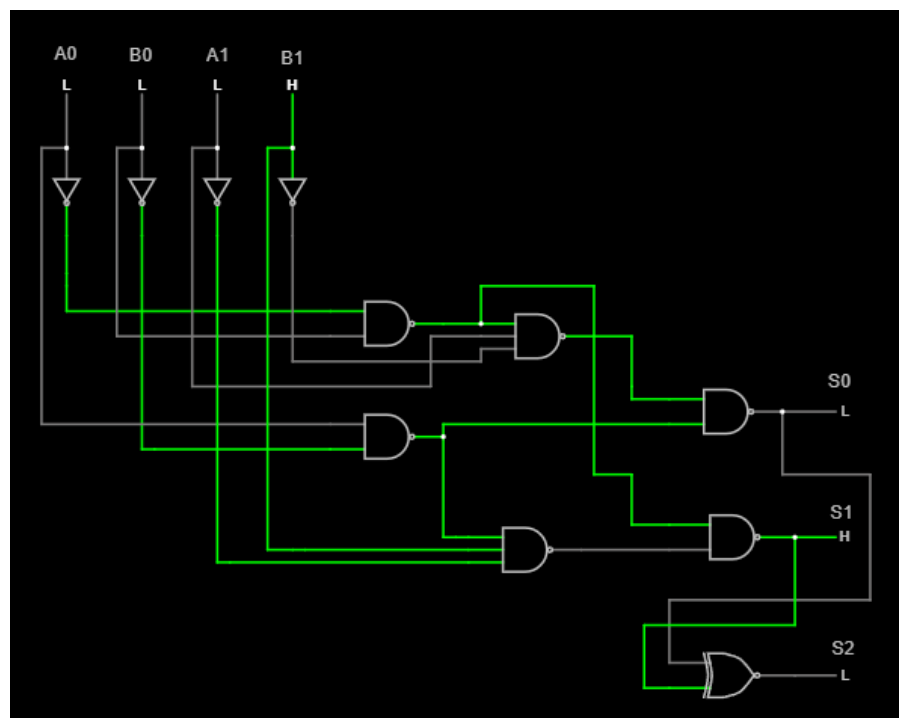


Figura 6: Circuito final implementado con compuertas NAND, NOT y XNOR.

Simplificación 2 en Falstad



Referencias

- [1] Simulación 1: Conversor BCD a Exceso-3. Disponible en: <https://tinyurl.com/27fbgv4n>
- [2] Simplificación 1: Conversor BCD a Exceso-3. Disponible en: <https://tinyurl.com/263xfme>
- [3] Simulación 2: Comparador binario. Disponible en: <https://tinyurl.com/26u3vja8>
- [4] Simplificación 2: Comparador binario. Disponible en: <https://tinyurl.com/2cpp9ej3>